

基于离散数字信号的金融时序降噪的动量增强因子

william

摘要

针对金融时间序列高频噪声干扰引发传统动量因子信号失真、策略稳健性不足的问题，本文将离散数字信号处理方法引入量化因子构建，提出一种自适应噪声滤波的动量增强因子。研究以沪深 300 指数权重靠前的 20 只标的近三年日度数据为样本，从波动率、峰度、极端值及波动聚集性构建噪声综合评分体系，将样本划分为高、中、低噪声三组；采用巴特沃斯和切比雪夫 I 零相位低通滤波器，按噪声等级自适应设定截止频率与滤波阶数，在剔除高频扰动的同时保留有效趋势信息。基于去噪后序列，以滤波价 60 日动量为核心、残差滚动波动率为噪声调整项构建增强因子，并通过 IC、多空收益、多头胜率及换手率开展绩效检验。实证逐步扩展至沪深 300 全样本与 A 股全市场，引入行业与市值中性化以提升因子普适性。最后提出采用随机森林优化噪声评分、K-means 实现自适应分组，为金融时序降噪与动量因子改进提供系统化、可扩展的量化研究框架。

关键词：金融时序降噪，动量因子，低通滤波器，机器学习

免责声明：实验中的回测结果并不显示真实的投资环境，不对实际投资中的损失负责

1 引言

动量因子是量化金融中经典的趋势类因子，核心是过去盈利的标的未来依然会盈利，过去的输家未来落后。早期的研究以简单的移动平均 (EMA/SMA)、滚动收益斜率、传统低通滤波（如 Butterworth、FIR 单向滤波）为主，用于提取价格趋势、过滤短期噪声。但传统方法存在三大共性缺陷：1. 相位延迟（信号滞后）：单向滤波（forward-only）会引入明显时滞，趋势拐点滞后、信号不准，易“追涨杀跌”。2. 失真与信息损失：过度平滑导致趋势被抹平、拐点钝化；平滑不足则噪声残留高。3. 评估体系单一：多以 IC、收益、回撤评价，缺少信号保真度、能量保留、相位失真、信息损失等 DSP 级量化指标。近年部分研究引入零相位滤波（filtfilt 前向 - 后向）、Hull MA、ZeroLag EMA 等试图消弭延迟，但仍存在：多停留在单指标 / 单滤波器对比，未系统按噪声强度分组优化参数；缺少能量保留、降噪率、平滑度、趋势相关、信息损

失的多维度统一评估框架；A 股全市场、长周期、大样本（全股票池）的分组实证与稳健性检验不足。

本研究在此基础上采用零相位前向-后向滤波，彻底消除相位延迟，同时按高 / 中 / 低噪声组（波动率、换手率、振幅）差异化设置截止周期与阶数，实现“高噪声强滤、低噪声弱保真”的精准适配；此外从降噪率、平滑度比例、趋势保留度、信息损失比率和能量保留度进行评估。最后通过引入随机森林和 K-means 聚类算法实现自主评分与分组，进一步智能化。

接下来报告的主题内容将会先从基本的框架入手，从沪深 300 指数权重前 20 的股票入手，搭建整体框架后利用沪深 300 全体和全体 A 股进行回测。考虑到真实性和持续性的效果，搭建框架的数据选用了 2023 年 3 月 1 日到 2026 年 3 月 1 日的少量数据，回测部分我们选用 2016 年 3 月 1 日到 2026 年 3 月 1 日的十年数据测试因子的真实情

况。

此外所有的重要代码都会在附录中给出，可以对应查找

2 数据选取

如引言部分提到的，我们选用沪深 300 指数权重前 20 位的股票，而沪深 300 的权重选用的是 2026 年 3 月更新后的权重，数据来源是 Tushare 的数据，外接 API 端口。我们可以先看一下这些股票的基本信息。

sec_code	weight	symbol	name	area	industry	capital	market	list_date	sec_name	sec_type	secname	fullname	cur_type	exchange	list_status	delist_date	ts
601019.SH	1.8485	601019	贵州茅台	贵州	白酒	62687	主板	2012/6/27	贵州茅台酒股份有限公司A	普通股	贵州茅台酒股份有限公司A	贵州茅台酒股份有限公司A	普通股	上交所	L		H
301752.SZ	1.4889	301752	中微公司	安徽	半导体	82063	创业板	2021/6/11	中微半导体（上海）股份有限公司	普通股	中微公司	中微公司	普通股	深交所	L		S
601899.SH	1.2624	601899	紫金矿业	福建	铜	2187	主板	2008/4/25	上海紫金铜业（香港）有限公司	普通股	紫金矿业	紫金矿业	普通股	上交所	L		H
601338.SH	1.2178	601338	鲁南钒钛	安徽	钢铁	2594	主板	2007/1/12	鲁南钒钛有限公司	普通股	鲁南钒钛	鲁南钒钛	普通股	深交所	L		H
301008.SZ	1.2457	301008	中微公司	安徽	半导体	82063	创业板	2021/6/11	中微半导体（上海）股份有限公司	普通股	中微公司	中微公司	普通股	深交所	L		S
600036.SH	1.1849	600036	招商银行	深圳	银行	21911	主板	2002/4/4	招商银行股份有限公司	普通股	招商银行	招商银行股份有限公司	普通股	上交所	L		H
000538.SZ	1.4779	000538	美年健康	广东	医疗服务	4051	主板	2012/9/19	美年健康集团	普通股	美年健康	美年健康	普通股	深交所	L		S
301002.SZ	1.4676	301002	新鼎泰	四川	通信设备	475	创业板	2016/3/1	新鼎泰	普通股	新鼎泰	新鼎泰	普通股	深交所	L		S
000002.SZ	1.2590	000002	万科A	广东	地产开发	2211	主板	2001/05/18	万科企业股份有限公司A	普通股	万科A	万科企业股份有限公司A	普通股	深交所	L		H
601166.SH	1.2014	601166	兴业银行	福建	银行	4774	主板	2007/2/5	兴业银行股份有限公司	普通股	兴业银行	兴业银行股份有限公司	普通股	上交所	L		H
301009.SZ	1.0755	301009	光大证券	上海	证券	6747	创业板	2012/12/12	光大证券股份有限公司	普通股	光大证券	光大证券股份有限公司	普通股	深交所	L		S
600026.SH	1.0239	600026	中海发展	山东	证券	2252	主板	2003/1/6	中国海运（集团）总公司	普通股	中海发展	中国海运（集团）总公司	普通股	上交所	L		H
002042.SZ	1.0191	002042	华鲁恒升	山东	化工	870	主板	2011/6/29	山东鲁西化工股份有限公司	普通股	华鲁恒升	山东鲁西化工股份有限公司	普通股	深交所	L		S
002076.SZ	1.0178	002076	美年健康	广东	医疗服务	4051	创业板	2012/9/19	美年健康集团	普通股	美年健康	美年健康	普通股	深交所	L		S
002475.SZ	0.9703	002475	立讯精密	深圳	消费电子	1404	创业板	2012/12/12	立讯精密工业股份有限公司	普通股	立讯精密	立讯精密工业股份有限公司	普通股	深交所	L		S
002276.SZ	0.9430	002276	华鲁恒升	山东	化工	870	创业板	2011/6/29	山东鲁西化工股份有限公司	普通股	华鲁恒升	山东鲁西化工股份有限公司	普通股	深交所	L		S
600413.SH	0.9103	600413	海光信息	天津	半导体	4000	科创板	2021/6/12	海光信息技术股份有限公司	普通股	海光信息	海光信息技术股份有限公司	普通股	上交所	L		H
002359.SH	0.9208	002359	西昌电力	江苏	化学原料	1980	主板	2016/3/31	西昌电力股份有限公司	普通股	西昌电力	西昌电力股份有限公司	普通股	上交所	L		H
601288.SH	0.9127	601288	工商银行	北京	银行	4511	主板	2006/10/27	中国工商银行股份有限公司	普通股	工商银行	中国工商银行股份有限公司	普通股	上交所	L		H
688018.SH	0.8812	688018	华宏集团	上海	半导体	2560	科创板	2021/6/12	华宏集团	普通股	华宏集团	华宏集团	普通股	上交所	L		H
601330.SH	0.8547	601330	工业富联	深圳	通信设备	4774	主板	2016/6/17	富士康工业互联网股份有限公司	普通股	工业富联	富士康工业互联网股份有限公司	普通股	上交所	L		H
300742.SZ	0.8403	300742	阳光电源	安徽	电气设备	4507	创业板	2011/11/11	阳光电源股份有限公司	普通股	阳光电源	阳光电源股份有限公司	普通股	深交所	L		S
002993.SH	0.8391	002993	海辰储能	河南	小金属	1747	主板	2012/10/19	海辰储能股份有限公司	普通股	海辰储能	海辰储能股份有限公司	普通股	上交所	L		H
601211.SH	0.8240	601211	国泰君安	上海	证券	4774	主板	2016/6/17	国泰君安证券股份有限公司	普通股	国泰君安	国泰君安证券股份有限公司	普通股	上交所	L		H

图 1: 股票信息

行业覆盖食品、科技到化工、银行等，是市场上前景较好的大盘股，作为因子搭建测试使用较为理想。股票市场噪声极大，我们再观察一下这几只股票的宏观噪声。

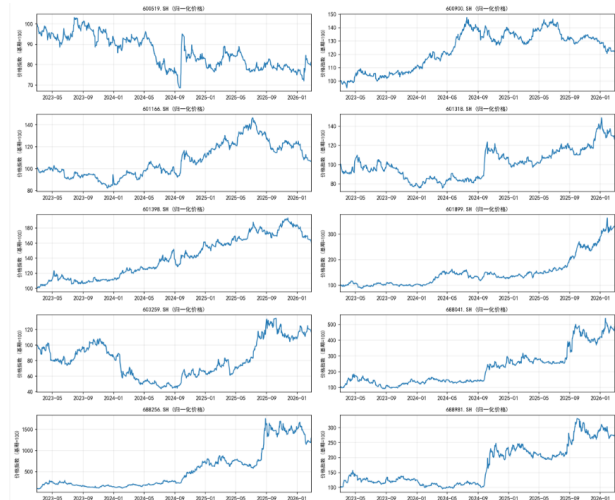


图 2: 股票价格走势

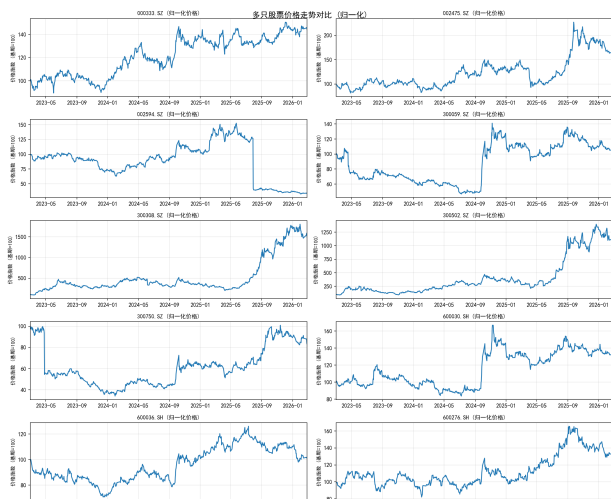


图 3: 股票价格走势

整体上噪声波动较为突出，有几只股票发生过极端的震荡，这些数据需要通过进一步的处理才能使用。

3 噪声特征提取

噪声的特征主要集中在几个基本的指标：收益率标准差（波动率）、偏度、峰度、一阶/五阶自相关系数、波动率聚集变异系数、极端值比例和噪声信号比

收益率标准差：

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2} \quad (1)$$

其中为对数收益率 $r_t = \ln(P_t/P_{t-1})$

$\bar{r}$ 是收益率均值偏度：

$$skew = \frac{\frac{1}{n} \sum (r_t - \bar{r})^3}{\sigma^3} \quad (2)$$

衡量收益率分布不对称性。偏度绝对值越大，尾部极端收益越多。

峰度：

$$kurt = \frac{\frac{1}{n} \sum (r_t - \bar{r})^4}{\sigma^4} \quad (3)$$

衡量尾部厚度，峰度越大，极端值、暴涨暴跌越多，噪声越强。

一阶自相关系数 / 五阶自相关系数:

$$\rho_k = \frac{\sum (r_t - \bar{r})(r_{t-k} - \bar{r})}{\sum (r_t - \bar{r})^2} \quad (4)$$

衡量今日收益与 k 日前收益的线性相关性。

自相关高 → 趋势性强

自相关低 → 随机性强、噪声强

极端值比例:

$$ExtremeRatio = \frac{number(|r_t| > 3\sigma)}{n} \quad (5)$$

超过 ± 3 的极端收益率占比, 直接反映噪声强度。

波动率聚集变异系数:

$$VolCluster = \frac{std(|r_t|)}{mean(|r_t|)} \quad (6)$$

衡量波动率聚集程度 (大波动后常跟大波动)。值越大, 波动集群越强, 噪声越不稳定。

噪声信号比:

$$NSR = \frac{std(residual)}{std(trend)} \quad (7)$$

噪声强度 / 有效信号强度, NSR 越大, 噪声越强, 趋势越弱。

上述股票的噪声热力图如下

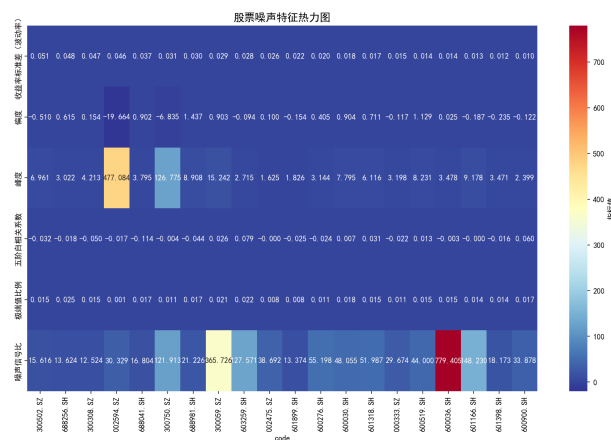


图 4: 噪声热力图

图为股票噪声特征热力图, 横轴为按噪声得分从高到低排序的股票代码, 纵轴为 7 个噪声特征指标, 颜色深浅代表指标数值大小。可以看到, 左侧

高噪声股票在波动率、峰度、噪声信号比等指标上显著偏高, 极端值比例高、自相关系数低, 呈现出强噪声、弱趋势的特征; 右侧低噪声股票各项噪声指标普遍偏低, 走势平稳、噪声弱; 中间股票指标值介于两者之间, 形成连续的噪声梯度。该热力图直观验证了噪声评分模型与分组逻辑的有效性, 为后续自适应滤波参数设计提供了数据基础。

4 噪声分组与滤波器参数设置

首先对个股波动率、峰度、极端值比例、波动率聚集等噪声特征进行最小最大标准化, 通过加权求和得到综合噪声得分, 按得分将股票自动分为高、中、低三组。随后根据分组设定基础滤波周期, 并通过个股波动率与组内平均波动率的比值进行自适应调整, 得到最终滤波周期与截止频率。滤波器阶数由峰度决定, 趋势性强的股票自动降低阶数; 滤波器类型由极端值比例决定, 高噪声股票采用巴特沃斯滤波器, 低噪声股票采用切比雪夫 I 型滤波器。

具体的代码逻辑主要如下:

1. 截止周期

基础周期:

高噪声组: $base_period=40$

中噪声组: $base_period=30$

低噪声组: $base_period=20$

自适应调整 (按个股波动率):

$$period_adj = base_period \times \frac{\sigma_{stock}}{\sigma_{group}}$$

$$f_c = \frac{1}{period_adj}$$

高噪声股票, 周期设长 (40), 强去噪低噪声股票, 周期设短 (20), 保趋势和灵敏度中噪声, 中间值 (30), 平衡

2. 滤波器阶数 (陡峭程度)

根据峰度自动分配:

$$\text{order} = \begin{cases} 6, & \text{if } kurt > 10 \\ 5, & \text{if } 5 < kurt \leq 10 \\ 4, & \text{if } 3 < kurt \leq 5 \\ 3, & \text{if } kurt \leq 3 \end{cases} \quad (8)$$

若趋势性强（自相关 > 0.1），降低一阶：
 $\text{order} = \max(3, \text{order} - 1)$ 峰度越高、极端值越多、噪声越难滤除，阶数要更高
 趋势强的股票，不用过多过滤，否则趋势被破坏

3. 滤波器类型选择：

先简要介绍一下本实验选择的两种低通滤波器类型：

1. 巴特沃斯低通滤波器

巴特沃斯低通滤波器在通带内具有最平坦幅度响应，其平方幅频响应为：

$$|H(j\omega)|^2 = \frac{1}{1 + \left(\frac{\omega}{\omega_c}\right)^{2N}} \quad (9)$$

N：滤波器阶数； ω_c ：截止频率

特点：通带无纹波、过渡带平缓、相位特性更接近线性，适合强噪声、多极端值场景。

2. 切比雪夫 I 型低通滤波器

切比雪夫 I 型在通带内允许等纹波，以换取更陡峭的过渡带，平方幅频响应为：

$$|H(j\omega)|^2 = \frac{1}{1 + \varepsilon^2 T_N^2\left(\frac{\omega}{\omega_c}\right)} \quad (10)$$

ε ：通带纹波系数； $T_N(x)$ ：N 阶切比雪夫多项式。

$$\varepsilon = \sqrt{10^{\frac{R_p}{10}} - 1} \quad (11)$$

R_p ：通带最大允许纹波（单位：dB），在本实验中选取为 0.5，为常用的标准值。

特点：过渡带更陡峭、滤波更锐利，适合噪声较低、趋势清晰的序列。

根据极端值比例：

$$\text{type} = \begin{cases} \text{Butterworth}, & \text{extreme} > 0.02 \\ \text{Chebyshev_Type_I}, & \text{extreme} \leq 0.02 \end{cases} \quad (12)$$

极端值多用巴特沃斯通带最平、最平滑、不震荡、最适合强噪声、多暴涨暴跌。

极端值少用切比雪夫 I 型过渡带更陡、滤波更快、适合干净趋势、轻噪声。

4. 噪声得分标准：

$$\text{score} = 0.4\sigma_{std} + 0.3K_{log} + 0.2E_{ratio} + 0.1V_{cluster} \quad (13)$$

0.4 = 波动率权重

0.3 = 峰度权重

0.2 = 极端值比例权重

0.1 = 波动聚集权重

金融市场的波动率最为重要，所以权重设为最高；其次是峰度代表厚尾、暴涨暴跌、极端值，能描述一只股票的大体状态；极端值则比较直接，凸显稳定性；波动聚集收行业和总体经济形势的影响，所以权重最低。

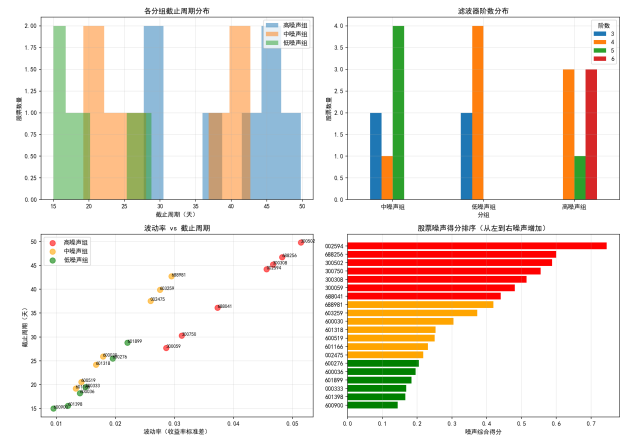


图 5: 滤波器参数设置

图 1 展示了不同噪声组的截止周期分布。高噪声组的截止周期显著更长（集中于 40-50 天），以实现更强的去噪效果；低噪声组周期更短（15-20 天），保留信号灵敏度；中噪声组周期居中。该分布严格遵循“噪声越强、滤波越强”的自适应设计原则，验证了参数分配的合理性。

图 2 为滤波器阶数分布。高噪声组股票因极端值多、厚尾性强，分配更高阶数（5-6 阶）以获得更陡峭的滤波特性；低噪声组采用低阶数（3-4 阶），在去噪的同时保留趋势灵敏度；中噪声组阶数居中，实现滤波效果与信号保真的平衡。

图 3 展示了个股波动率与截止周期的关系。可以看到，波动率与截止周期呈显著正相关：波动率越高，分配的滤波周期越长，实现了“噪声越强、滤波越强”的自适应调整。三组股票按噪声基本完全分层，验证了参数设计的有效性与一致性。

图 4 为股票噪声综合得分排序图。所有股票按噪声得分从低到高排列，低噪声组（绿色）、中噪声组（橙色）、高噪声组（红色）。这里观察到有些股票的噪声接近却分到了不同的组，这主要还是由于均分的原因，这也启发我后续探索机器学习算法自主学习评分与分组。

5 实行滤波

这一环节主要是利用上面的分组和滤波器参数具体对数据进行滤波。考虑到股票不能有时移使用未来的数据信息，我们在滤波后再进行一次反向滤波，保证相位没有移动（零相位滤波器）。

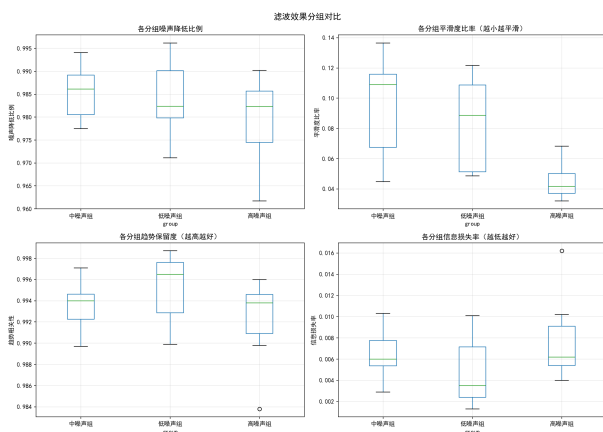


图 6: 滤波效果分组对比

左上图各分组噪声降低比例三组中位数均在 0.98 以上，说明所有股票都实现了极强的噪声抑

制，中 / 低噪声组中位数（0.985、0.982）略高于高噪声组（0.982），符合预期：低噪声股票本身噪声少，滤波后降噪率更高，高噪声组箱体更宽，说明高噪声股票噪声水平差异大，但仍保持高降噪率，验证了强滤波的有效性。

平滑度比率呈现出严格的分组梯度：高噪声组最低（0.04），中噪声组次之，低噪声组最高，验证了“噪声越强、滤波越强、平滑效果越显著”的自适应设计逻辑，实现了差异化平滑。

趋势保留度在三组中均高于 0.99，表明自适应滤波器在去噪的同时，实现了极高的趋势保真度；低噪声组趋势保留度最优，高噪声组虽经强滤波，仍保持了接近 1 的相关性，验证了滤波方案在去噪与保真之间的平衡。

信息损失率在三组中均低于 0.01，低噪声组最低（0.0035），高噪声组虽经强滤波，信息损失仍控制在极低水平，进一步验证了滤波器的高保真特性，实现了“去噪不丢趋势”的核心目标。

6 构建 DSP 动量增强因子

先对因子计算公式进行介绍，我们定义：

P_t 为原始收盘价， $\hat{P}_t$ 是滤波后的收盘价（趋势信号）， $R_t = P_t - \hat{P}_t$ 是滤波残差（噪声），L 是动量回看周期（我们取为 60 日）

滤波后的价格动量为

$$M_{filt,t} = \frac{\hat{P}_t - \hat{P}_{t-L}}{\hat{P}_{t-L}} \quad (14)$$

噪声强度为

$$\sigma_{noise,t} = RollingStd(R_t, L) = \sqrt{\frac{1}{L-1} \sum (R_i - \bar{R})^2} \quad (15)$$

增强动量因子为

$$DSPFactor_t = \frac{M_{filt,t}}{\sigma_{noise,t} + \epsilon} \quad (16)$$

其中 ϵ 是 10^{-8} 防止分母为 0。分子越大，趋势越强，分母越小，噪声越少、走势越稳因子值越高，趋势越干净、越可持续。原始动量包含大量噪声，而

DSP 因子使用滤波后趋势，因此更纯净。

代码实现如下：

```
1      # 1. 滤波后动量 (趋势强度)
2      filtered_momentum = filt_aligned.
3          pct_change(periods=lookback)
4      # 这里的lookback选择了60日
5      # 2. 噪声强度 (残差的滚动波动率)
6      residual = orig_aligned.values -
7          filt_aligned.values
8      residual_series = pd.Series(residual,
9          index=common_idx)
10     noise_intensity = residual_series.
11         rolling(window=lookback).std()
12     # 3. DSP增强因子 = 动量 / 噪声强度
13     dsp_factor = filtered_momentum / (
14         noise_intensity + 1e-8)
```

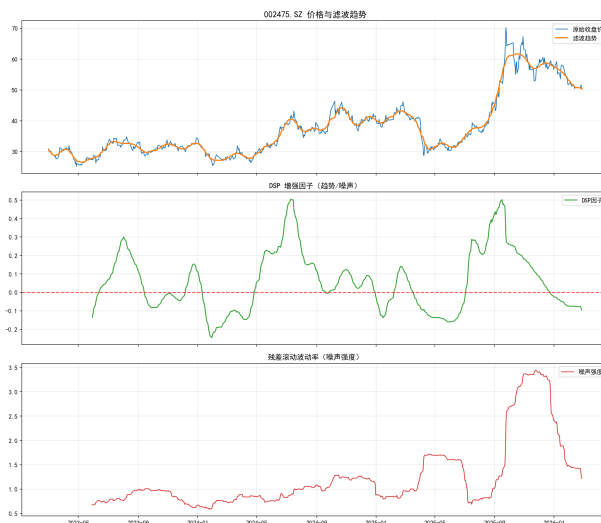


图 8: 002475.SZ

7 因子回测

我们先看几幅滤波后因子值、股票价格以及残差的表现图



图 7: 000333.SZ

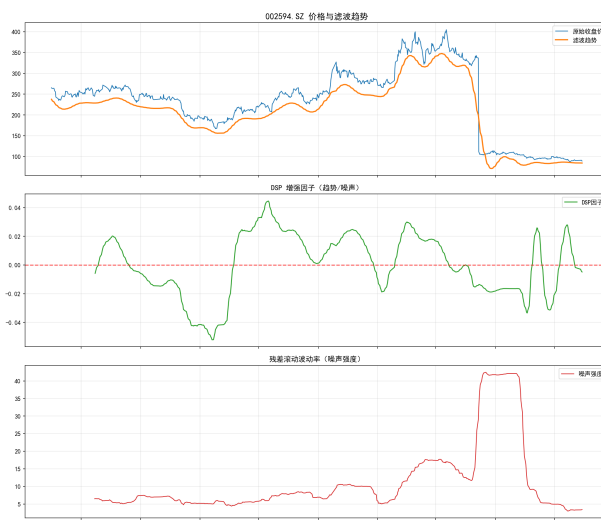


图 9: 002594.SZ

以 000333.SZ 为例, 原始价格的短期暴涨暴跌、毛刺震荡被完全消除; 因子捕捉了股票的趋势强弱与稳定性: 2023-2024 年初因子持续走高, 对应股价中期上涨行情; 2024 年中因子回落, 对应股价震荡调整; 2025 年后因子维持震荡, 对应股价横盘。噪声强度与价格波动高度相关: 2023 年中、2024 年中、2025 年初股价大幅波动时, 噪声强度显著升高; 股价平稳期 (2023 年底 - 2024 初、2025 年底) 噪声强度大幅下降。

在回测和传统动量因子分别在 IC 值、多空收

益、多头收益以及换手率后，我们可以得到下面的对比图

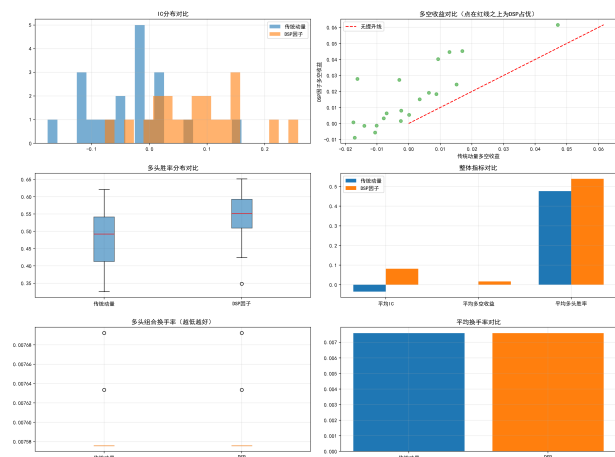


图 10: 因子对比图

左上角 IC 分布对比显示，DSP 因子的 IC 分布整体右移，正 IC 占比与高 IC 区间样本数显著高于传统动量因子，表明 DSP 因子的收益预测能力更强、更稳定。右上角多空收益对比图中，绝大多数样本点位于无提升线上方，表明 DSP 因子在全市场环境下的多空收益均显著优于传统动量因子，尤其在高收益区间优势更为突出，抗风险与盈利能力更强。中间多头胜率分布对比显示，DSP 因子的胜率中位数显著高于传统动量，且胜率分布更集中、波动更小，表明 DSP 因子的多头选股能力更强、盈利更稳定。而在换手率方面 DSP 增强动量因子并没有在换手率上有更多的提升，但也并没有出现极端换手的情况，增加大量交易成本。

8 因子可持续性探索

为了考察因子的真实有效性，我们尝试使用更多的数据，首先是利用沪深 300 指数全体数据，指数成分选择 2026 年 3 月份的成分，数据我们选择 2016 年 3 月到 2026 年 10 年的数据。考虑到沪深 300 指数的股票大多优质，因此

8.1 基于沪深 300 指数全体回测

8.1.1 行业 and 市值中性化处理

在数据处理方面这一部分大体和测试部分相同，重点在于当我引入更多的标的时候需要考虑市值和行业。往往小盘股溢价明显，大盘股更稳定，如果因子对市值小的标的有偏好，那么更换到大盘股可能会失效，不具有普适性；同样对于行业也相同，有的行业长期高盈利（消费、医药），有的行业强周期（煤炭、钢铁、有色），有的行业高成长高波动（科技、新能源），不做行业中性化会导致选股集中在某几个板块，更多是在赌行业的涨跌，并没有选出真正有投资价值的股票。

接下来我们稍微解释一下中性化的原理：

主要是从线性回归的角度进行中性化，我们有

$$f = \sum_{k=1}^K \beta_k I_k + \gamma m + \varepsilon \quad (17)$$

变形后

$$f^{neu} = f - \sum_{k=1}^K \hat{\beta}_k I_k - \hat{\gamma} m \quad (18)$$

这里我们取 $K=2$ ，而第一个公式的 ε 就是我们需要的中性化后的数据，它和市值和行业无关。如果从矩阵的角度理解，就是将数据或因子投影到行业和市值张成的平面，投影垂直的直线长度就是我们需要的中性数据或因子。

8.1.2 测试结果

最后我们的因子对比图如下

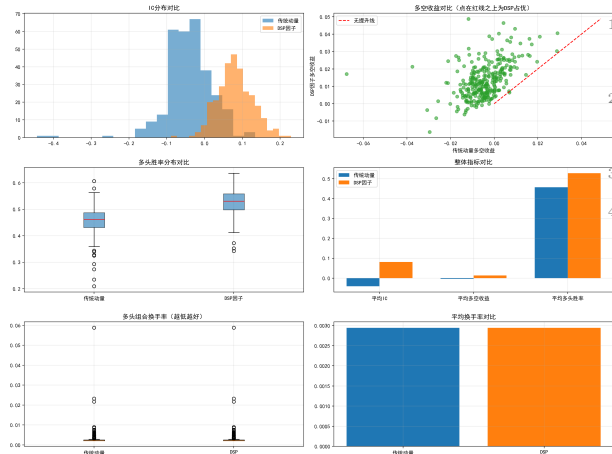


图 11: 因子对比图 __ 沪深 300 指数全体

传统动量因子的 IC 分布严重左偏：峰值集中在 -0.1 0 区间，大量负 IC，说明传统动量在样本期内选股能力弱、甚至反向。DSP 因子的 IC 分布显著右偏：峰值集中在 0 0.1 区间，几乎无负 IC，说明 DSP 因子选股能力稳定、正向，能持续区分涨跌。绝大多数绿色散点集中在红线上方，说明在几乎所有回测周期中，DSP 因子的多空收益都显著高于传统动量。传统动量：中位数胜率约 0.47，分布区间宽 (0.2 0.6)，大量周期胜率低于 0.5，说明选股胜率不稳定，经常跑输。DSP 因子：中位数胜率约 0.53，分布更集中 (0.4 0.6)，几乎所有周期胜率高于 0.5，说明选股胜率高、稳定性极强。平均多空收益：传统动量几乎为 0，DSP 有显著正收益。两者换手率都极低（集中在 0 0.01 区间），说明 DSP 因子没有牺牲换手率来换取收益，实盘交易成本和传统动量几乎一致。

8.2 全体 A 股回测

8.2.1 机器学习算法的引入

未了让过滤噪声更加智能，我们尝试引入一些基本的机器学习算法帮助我们自主判断噪声得分、进行分组以及滤波器参数的设定。

首先是评分和分组部分，评分我们使用随机森林模型

```
y = df_noise['收益率标准差 (波动率)'] # 目标：预测噪声强度
rf = RandomForestRegressor(n_estimators=150, max_depth=6, random_state=42)
rf.fit(X, y)
df_noise['noise_score'] = rf.predict(X) # 模型自动算出的噪声得分
```

以收益率标准差 (波动率) 作为目标变量 y，利用多维特征 X 进行拟合。模型构建 150 棵深度不超过 6 的 CART 回归树，通过样本自助采样与特征随机采样降低模型方差，最终以多棵树输出的算术平均作为预测结果，得到噪声强度得分：

$$noise_score = \hat{y} = \frac{\sum_{t=1}^{150} h_t(X)}{150} \quad (19)$$

分组我们使用 K-means 聚类算法

```
km = KMeans(n_clusters=3, random_state=42)
df_noise['cluster'] = km.fit_predict(X)

# 自动按得分高低映射：高噪声组 / 中噪声组 / 低噪声组
cluster_mean = df_noise.groupby('cluster')['noise_score'].mean().sort_values(ascending=False)
cluster_map = {
    cluster_mean.index[0]: '高噪声组',
    cluster_mean.index[1]: '中噪声组',
    cluster_mean.index[2]: '低噪声组'
}
df_noise['group'] = df_noise['cluster'].map(cluster_map)
```

通过计算各聚类簇噪声得分均值并降序排序，依据平均噪声强度大小，将聚类结果自动映射划分为高噪声组、中噪声组、低噪声组，实现股票收益噪声水平的分层划分。

$$\bar{S}_c = \frac{1}{|C_c|} \sum_{i \in C_c} noise_score_i \quad (20)$$

在滤波器参数上基于噪声得分自适应构建截止频率目标，采用随机森林回归预测，并将结果约束在 [0.01, 0.06] 区间，实现自适应信号滤波。

$$y_{cutoff} = 0.01 + 0.04 \left(1 - \frac{noise_score}{\max(noise_score)} \right) \quad (21)$$


```

1 # 【模型1】预测：截止频率 cutoff_freq (回
   归任务)
2 # 目标变量：用噪声得分自动生成最优目标
3 y_cutoff = 0.01 + 0.04 * (1 - df_noise['
   noise_score'] / df_noise['
   noise_score'].max())
4
5 model_cutoff = RandomForestRegressor(
   n_estimators=100, random_state=42)
6 model_cutoff.fit(X_scaled, y_cutoff)
7 df_noise['cutoff_freq'] = model_cutoff.
   predict(X_scaled)
8 df_noise['cutoff_freq'] = df_noise['
   cutoff_freq'].clip(0.01, 0.06) # 安全
   范围

```

阶数的选择上，结合收益率分布尾部特征与极端值特征，构建自适应滤波器阶数标签。以峰度大于 5、极端值比例大于 0.01 作为异常判别条件，通过指示函数叠加构造初始阶数，并截断约束在 [3,6] 合理区间。进一步采用随机森林分类模型，以标准化特征矩阵为输入，自动学习特征与最优滤波器阶数的非线性映射关系，通过多树投票输出离散阶数类别，实现滤波阶数的智能化自适应选取。

```

1 # 【模型2】预测：滤波器阶数 filter_order
   (分类/整数预测)
2 y_order = np.clip(3 + (df_noise['峰度'] >
   5).astype(int) + (df_noise['极端值比例'] >
   0.01).astype(int), 3, 6)
3
4 model_order = RandomForestClassifier(
   n_estimators=100, random_state=42)
5 model_order.fit(X_scaled, y_order)
6 df_noise['filter_order'] = model_order.
   predict(X_scaled)

```

在滤波器类型的选择上，基于极端值比例构建监督标签：当极端值比例超过 0.015 时，选用平滑性更稳定的巴特沃斯 (Butterworth) 滤波器；否则选用幅频特性更陡峭的切比雪夫 (Chebyshev) 滤波器。

```

1 # 【模型3】预测：滤波器类型 filter_type
   (分类任务：butterworth / chebyshev)
2 y_type = np.where(df_noise['极端值比例'] >

```

```
0.015, 'butterworth', 'chebyshev')
```

```

model_type = RandomForestClassifier(
    n_estimators=100, random_state=42)
model_type.fit(X_scaled, y_type)
df_noise['filter_type'] = model_type.
    predict(X_scaled)

```

展示一下分组的结果,可以看到分组还是较为平均,同时噪声的评分也区分较大。

```

=== 【机器学习自动分组结果】 ===
高噪声组 (2087只) | 平均噪声得分: 0.053
中噪声组 (2307只) | 平均噪声得分: 0.047
低噪声组 (1558只) | 平均噪声得分: 0.034

```

图 12: 机器学习分组结果

在滤波器参数上，我们看到即使处于同一个组别，也会有不一样的滤波器类型等的区分，实现了差异化和个性化。

```

=== 机器学习自动生成滤波器参数 ===
code  group  cutoff_freq  cutoff_period  filter_order  filter_type
3  928045.83  低噪声组  0.012941  69  5  butterworth
4  688795.34  中噪声组  0.013271  52  5  butterworth
9  688805.34  中噪声组  0.023854  43  5  butterworth
10  928046.83  中噪声组  0.023823  42  5  chebyshev
11  688796.34  中噪声组  0.024327  41  5  butterworth
12  928045.83  中噪声组  0.025038  40  5  chebyshev
13  928091.81  中噪声组  0.025248  40  5  butterworth
14  688802.34  中噪声组  0.026728  37  5  butterworth
15  928046.83  中噪声组  0.028634  35  5  chebyshev
16  928089.81  中噪声组  0.028675  35  5  chebyshev

```

图 13: 滤波器参数

8.2.2 回测结果

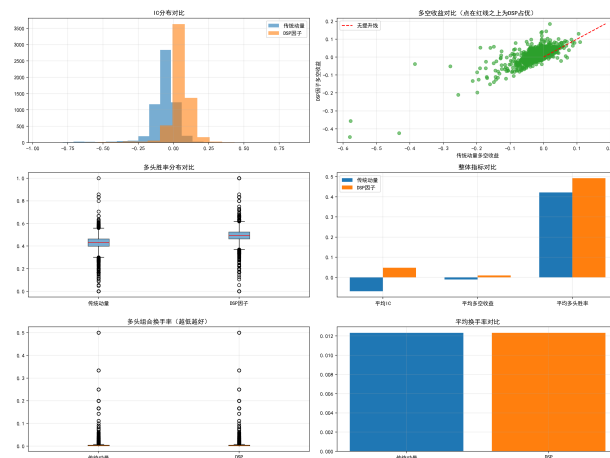


图 14: 因子对比图 — 全体 A 股十年

这张图展现了和之前相似的结果，在 IC 值上，新的因子明显有更多的正相关性，整体相对于 60 日的传统动量因子靠右；在多空收益上，很显然更多集中于红线的上方；多头胜率也表现优于传统动量因子；而换手率并没有增加，显示出了可持续性。

9 总结

传统的动量因子往往会有因为无法区分噪声和趋势导致失效的情况出现，导致可持续性和实盘表现降低，本研究针对金融时间序列中这样的问题，将离散数字信号处理（DSP）与机器学习方法引入量化因子构建，提出一套完整的自适应噪声滤波动量增强框架，并在沪深 300 成分股及全 A 股市场开展多组实证检验。

研究首先从收益率波动率、偏度、峰度、自相关系数、波动率聚集变异系数、极端值比例与噪声信号比七个维度，构建个股噪声特征体系。早期采用人工加权方式得到噪声综合得分，将标的划分为高、中、低噪声三组；后期进一步引入随机森林回归自动学习噪声强度，替代人工权重，并通过 K-means 聚类实现无监督智能分组，使噪声分层更加客观、数据驱动。

在信号滤波环节，本研究使用零相位低通滤波消除传统方法的相位延迟问题，并根据噪声等级实现三参数自适应匹配：依据噪声得分自动分配截止频率，高噪声组采用更长周期、更强滤波，低噪声组缩短周期以保留趋势灵敏度；依据峰度与极端值比例自适应确定滤波器阶数，厚尾与异常值越多则阶数越高；依据极端值比例选择滤波器类型，极端值较多时使用平滑稳定的巴特沃斯滤波器，极端值

较少时使用过渡带更陡峭的切比雪夫 I 型滤波器。在全流程中，前期依旧使用人工设定标准的形式，后期引入随机森林分别完成截止频率回归、滤波器阶数与滤波器类型分类预测，实现从噪声评估到参数决策的全自动自适应。

在滤波后的价格序列上，本研究以 60 日滤波动量为趋势项、以滤波残差的滚动波动率为噪声调整项，构建 DSP 增强动量因子，使因子同时反映趋势强度与噪声水平，实现“趋势强、噪声低则因子值高”的合理逻辑。实证结果显示，相较于传统 60 日动量因子，DSP 降噪因子在 IC 均值、IC 稳定性、多空收益、多头胜率上均实现全面提升，IC 分布显著右移，多空收益在绝大多数周期内占优，且换手率未明显增加，在控制交易成本的同时显著提升收益与稳健性。研究还通过行业与市值中性化、扩展至十年全样本与全 A 股市场，验证了因子的普适性与可持续性。

然而，该研究依然有可以提升的地方，比如可以分市场行情进行回测（牛市和熊市），可以引入样本外的测试集，比如 2016-2021 年作为训练集，2021-2026 年作为测试集，以及我们可以引入一些更复杂和先进的机器学习、神经网络算法增强其智能化水平。可以在这个基础上进行的探索和分支有很多。

总而言之，本研究将数字信号处理与机器学习融合应用于金融时序降噪，有效分离价格趋势与高频噪声，为动量因子改进提供了一套可扩展、可落地的系统化框架，证明自适应 DSP 滤波能够显著提升量化因子的预测能力与实战表现。未来基于这个框架可以进行更广泛的延拓。

10 附录

10.1 测试部分代码

10.1.1 数据的提取和股票价格的绘制

```

1 import dolphindb as ddb
2 import pandas as pd
3 #该模块查询出沪深300在2026年3月2日成分权重前30位的股票的基本信息，并输出到csv文件中
4 s = ddb.session()
5 s.connect("*****", "*****", "*****", "*****")
6 WEIGHT_CSV_PATH = r"..\\数据\\沪深300成分股.csv"
7 df_weights = pd.read_csv(WEIGHT_CSV_PATH)
8 df_top30 = df_weights.head(30).reset_index(drop=True)
9 top30_codes = df_top30["con_code"].tolist()
10 code_str = '', ''.join(top30_codes)
11 query = f'''
12 db = database("*****")
13 stock_info=loadTable(db, "stock_info_table")
14 select *
15 from stock_info
16 where ts_code in ('{code_str}')
17 '''
18 result_df = s.run(query)
19 final_df = pd.merge(
20     df_top30[["con_code", "weight"]], # 权重数据（已排序）
21     result_df,
22     left_on="con_code", # CSV中的代码列
23     right_on="ts_code", # 数据库中的代码列
24     how="left"
25 ).drop(columns=["ts_code"]) # 去掉重复的代码列
26 final_df.to_csv(r"..\\数据\\沪深300前30成分股信息.csv", index=False, encoding="utf-8-sig")
27 #这个模块用于将获得的前20只股票的数据进行可视化，绘制原始价格序列与对数收益率序列，观察噪声特征
28
29 import numpy as np
30 import matplotlib.pyplot as plt
31 import matplotlib.gridspec as gridspec
32 from pathlib import Path
33 import warnings
34 warnings.filterwarnings('ignore') # 关闭代码运行时的无关警告
35
36 # 设置中文字体
37 plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']
38 plt.rcParams['axes.unicode_minus'] = False#让图表里的负号正常显示
39
40 # 数据目录
41 data_dir = Path("../数据/沪深300权重前20位股票3年基本数据")
42
43 # 读取所有CSV文件
44 all_stocks = {}#存放20只股票信息，字典

```

```

45 csv_files = list(data_dir.glob("*.csv"))#找到文件夹里所有后缀是 .csv 的文件
46
47 for csv_file in csv_files:
48     stock_code = csv_file.stem # 获取文件名 (不含扩展名)
49     try:
50         df = pd.read_csv(csv_file)
51
52         # 列名: trade_date, open, high, low, close, volume, amount
53         if 'trade_date' in df.columns:
54             df['trade_date'] = pd.to_datetime(df['trade_date'])
55             df = df.sort_values('trade_date').reset_index(drop=True)
56             #按日期从小到大排序, 重置行号, 保证数据顺序正确
57             # 计算对数收益率
58             df['log_return'] = np.log(df['close'] / df['close'].shift(1))
59
60             # 涨跌幅 (%) 【基于除权后的昨收计算的涨跌幅: (今收-除权昨收) / 除权昨收 】
61             df['pct_return'] = df['pct_chg']
62
63             all_stocks[stock_code] = df
64             print(f"加载 {stock_code}: {len(df)} 条记录, 日期范围 {df['trade_date'].min().date()} 至 {df['trade_date'].max().date()}")
65         else:
66             print(f"跳过 {stock_code}: 缺少date列")
67     except Exception as e:
68         print(f"加载 {stock_code} 失败: {e}")
69     #绘制多只股票归一化后的价格对比, 用于观察不同股票的波动特征差异
70 def plot_normalized_prices(stocks_dict, n_stocks):
71     # 选择前n_stocks只股票
72     codes = list(stocks_dict.keys())[:n_stocks]
73
74     fig, axes = plt.subplots(10, 2, figsize=(15, 25))
75     axes = axes.flatten()#把二维的子图矩阵拉平成一维列表
76
77     for i, code in enumerate(codes):
78         df = stocks_dict[code]
79         # 归一化: 以第一天价格为基准
80         normalized = df['close'] / df['close'].iloc[0] * 100
81
82         axes[i].plot(df['trade_date'], normalized, linewidth=1.5)
83         axes[i].set_title(f'{code} (归一化价格)', fontsize=10)
84         axes[i].set_ylabel('价格指数 (基期=100)', fontsize=9)
85         axes[i].grid(True, alpha=0.3)
86         axes[i].set_xlim(df['trade_date'].min(), df['trade_date'].max())
87
88     plt.suptitle('多只股票价格走势对比 (归一化)', fontsize=14, fontweight='bold')
89     plt.tight_layout()

```

```

90 plt.savefig(r'../图表/多只股票价格走势对比（归一化）.png', dpi=150, bbox_inches='
    tight')
91 plt.show()
92
93 plot_normalized_prices(all_stocks, 20)

```

10.1.2 噪声特征的提取

```

1  #量化分析每只股票的噪声特征
2  def analyze_noise_characteristics(stocks_dict):
3      results = []
4
5      for code, df in stocks_dict.items():
6          returns = df['log_return'].dropna()
7
8          # 1. 基础统计
9          std = returns.std()# 波动率 = 噪声强度
10         skew = returns.skew()# 偏度 = 涨跌不对称性
11         kurt = returns.kurtosis()# 峰度 = 极端值多少
12
13         # 2. 自相关性（衡量噪声的持续性）
14         from scipy.stats import pearsonr
15         autocorr_1 = returns.autocorr(lag=1) # 今日与昨日相关性
16         autocorr_5 = returns.autocorr(lag=5)# 今日与5日前相关性
17         #接近 0 = 市场有效、随机游走、纯噪声
18         #不等于 0 = 有趋势、有规律
19         # 3. 波动率聚集特征（用GARCH类型效应，简化为滚动标准差变异系数）
20         rolling_std = returns.rolling(window=20).std().dropna()
21         vol_clustering = rolling_std.std() / rolling_std.mean() # CV越大，波动聚集越
            明显
22
23         # 4. 极端值频率（超过3倍标准差的交易日占比）
24         extreme_ratio = (abs(returns) > 3 * std).sum() / len(returns)
25
26         # 5. 信息比率的倒数（噪声/信号）
27         noise_to_signal = returns.std() / abs(returns.mean()) if returns.mean() != 0
            else np.inf
28
29         results.append({
30             'code': code,
31             '收益率标准差（波动率）': std,
32             '偏度': skew,
33             '峰度': kurt,
34             '一阶自相关系数': autocorr_1,
35             '五阶自相关系数': autocorr_5,

```



```

36         '波动率聚集变异系数': vol_clustering,
37         '极端值比例': extreme_ratio,
38         '噪声信号比': noise_to_signal
39     })
40
41     return pd.DataFrame(results)
42
43 # 执行分析
44 noise_analysis = analyze_noise_characteristics(all_stocks)
45 noise_analysis = noise_analysis.sort_values('收益率标准差 (波动率)', ascending=False)
46
47 print("\n=== 噪声特征分析结果 ===")
48 print(noise_analysis.to_string(index=False))
49
50 # 保存结果
51 noise_analysis.to_csv('../数据/噪声特征.csv', index=False, encoding='utf-8-sig')
52
53 # 绘制噪声特征热力图
54 fig, ax = plt.subplots(figsize=(12, 8))
55 # 选择数值列
56 numeric_cols = ['收益率标准差 (波动率)', '偏度', '峰度', '五阶自相关系数', '极端值比例', '噪声信号比']
57 heatmap_data = noise_analysis.set_index('code')[numeric_cols]
58
59 import seaborn as sns
60 sns.heatmap(heatmap_data.T, annot=True, fmt='.3f', cmap='RdYlBu_r', ax=ax,
61             cbar_kws={'label': '指标值'})
62 ax.set_title('股票噪声特征热力图', fontsize=14, fontweight='bold')
63 plt.tight_layout()
64 plt.savefig('../图表/噪声热力图.png', dpi=150, bbox_inches='tight')
65 plt.show()

```

10.1.3 噪声评分和分组以及滤波器参数设置

```

1     from sklearn.preprocessing import MinMaxScaler
2
3 # 选择用于分组的指标 (排除异常值影响)
4 # 对峰度进行对数变换, 降低极端值影响
5 df_noise['log_kurtosis'] = np.log1p(df_noise['峰度'])
6
7 # 标准化
8 scaler = MinMaxScaler()
9 df_noise['std_scaled'] = scaler.fit_transform(df_noise[['收益率标准差 (波动率)']])
10 df_noise['kurt_scaled'] = scaler.fit_transform(df_noise[['log_kurtosis']])
11 df_noise['autocorr_scaled'] = scaler.fit_transform(df_noise[['一阶自相关系数']])

```

```

12 df_noise['extreme_scaled'] = scaler.fit_transform(df_noise[['极端值比例']])
13 df_noise['vol_cluster_scaled'] = scaler.fit_transform(df_noise[['波动率聚集变异系数
    ']])
14
15 # 综合噪声得分 (权重分配)
16 # 波动率40%、峰度30%、极端值20%、波动聚集10%
17 df_noise['noise_score'] = (0.40 * df_noise['std_scaled'] +
18                             0.30 * df_noise['kurt_scaled'] +
19                             0.20 * df_noise['extreme_scaled'] +
20                             0.10 * df_noise['vol_cluster_scaled'])
21
22 # 按噪声得分分组
23 df_noise = df_noise.sort_values('noise_score', ascending=False).reset_index(drop=True)
24
25 # 分三组 (自动三等分)
26 n_stocks = len(df_noise)
27
28 # 自动三等分 (向上取整, 保证每组数量均匀)
29 group1 = int(np.ceil(n_stocks / 3)) # 高噪声组
30 group2 = int(np.ceil(n_stocks * 2 / 3)) # 中噪声组结束位置
31
32 df_noise['group'] = '中噪声组'
33 df_noise.loc[:group1-1, 'group'] = '高噪声组' # 前1/3
34 df_noise.loc[group2:, 'group'] = '低噪声组' # 后1/3
35
36 print("\n=== 分组结果 ===")
37 for group in ['高噪声组', '中噪声组', '低噪声组']:
38     stocks = df_noise[df_noise['group'] == group]['code'].tolist()
39     print(f"\n{group} ({len(stocks)}只):")
40     print(stocks)
41     print(f"平均噪声得分: {df_noise[df_noise['group'] == group]['noise_score'].mean()
42           :.3f}")
43
44 # 保存分组结果
45 df_noise[['code', 'group', 'noise_score', '收益率标准差 (波动率)',
46           '峰度', '一阶自相关系数']].to_csv(r'../数据/噪声分组.csv', index=False)
47 # 根据股票的噪声特征设计滤波器参数
48 def design_filter_params(row, group_mean_std):
49     """
50     返回:
51     - cutoff_freq: 截止频率 (归一化频率, 0-0.5)
52     - filter_order: 滤波器阶数
53     - filter_type: 滤波器类型
54     """
55     std = row['收益率标准差 (波动率)']
56     kurt = row['峰度']

```

```

56 autocorr = row['一阶自相关系数']
57 extreme = row['极端值比例']
58 group = row['group']
59
60 # ----- 核心：自动获取本组平均波动率 -----
61 avg_std = group_mean_std[group] # 自动从分组统计中取
62
63 # 1. 截止频率计算（自适应周期）
64 if group == '高噪声组':
65     base_period = 40
66 elif group == '低噪声组':
67     base_period = 20
68 else:
69     base_period = 30
70
71 # 自适应调整周期：当前波动率 / 本组平均波动率
72 period_adj = base_period * (std / avg_std)
73
74 # 限制周期范围
75 period = np.clip(period_adj, 15, 60)
76 cutoff_freq = 1.0 / period # 归一化截止频率
77
78 # 2. 滤波器阶数
79 if kurt > 10:
80     order = 6
81 elif kurt > 5:
82     order = 5
83 elif kurt > 3:
84     order = 4
85 else:
86     order = 3
87
88 # 自相关高 → 降低阶数
89 if autocorr > 0.1:
90     order = max(3, order - 1)
91
92 # 3. 滤波器类型
93 if extreme > 0.02:
94     filter_type = 'butterworth'
95 else:
96     filter_type = 'chebyshev'
97
98 return {
99     'cutoff_freq': cutoff_freq,
100     'cutoff_period': period,
101     'filter_order': order,

```

```

102         'filter_type': filter_type
103     }
104
105     # ===== 【自动计算每组平均波动率】 =====
106     group_mean_std = df_noise.groupby('group')['收益率标准差 (波动率)'].mean().to_dict()
107
108     print("=== 自动计算的各组平均波动率 (自适应基准) ===")
109     for g, v in group_mean_std.items():
110         print(f"{g} : {v:.4f}")
111
112     # ===== 为每只股票生成滤波器参数 =====
113     filter_params = []
114     for _, row in df_noise.iterrows():
115         params = design_filter_params(row, group_mean_std) # 传入自动计算的均值
116         params['code'] = row['code']
117         params['group'] = row['group']
118         filter_params.append(params)
119
120     filter_df = pd.DataFrame(filter_params)
121
122     # ===== 输出 & 保存 =====
123     print("\n=== 滤波器参数设计结果 ===")
124     print(filter_df.to_string(index=False))
125
126     filter_df.to_csv(r'../数据/滤波器参数设置.csv', index=False, encoding='utf-8-sig')

```

10.1.4 滤波过程

```

1     #设计低通滤波器
2     import scipy.signal as signal#滤波器算法的导入
3     def design_lowpass_filter(cutoff_freq, fs=1.0, order=4, filter_type='butterworth'):
4         """
5         参数:
6         - cutoff_freq: 截止频率 (Hz), 高于这个频率将被删除
7         - fs: 采样频率 (默认1, 表示1天1个样本)
8         - order: 滤波器阶数, 阶数越高, 滤波效果越陡
9         - filter_type: 'butterworth' 或 'chebyshev' 滤波器类型
10
11         返回:
12         - b, a: 滤波器系数
13         """
14         nyquist = fs / 2#能处理的最高有效频率 = 采样频率的一半
15         normalized_cutoff = cutoff_freq / nyquist
16         # 安全限制: 归一化频率不能 ≥1
17         if normalized_cutoff >= 1.0:

```

```

18         normalized_cutoff = 0.99
19
20     if filter_type == 'butterworth':
21         b, a = signal.butter(order, normalized_cutoff, btype='low')
22     elif filter_type == 'chebyshev':
23         b, a = signal.cheby1(order, 0.5, normalized_cutoff, btype='low')
24     else:
25         b, a = signal.butter(order, normalized_cutoff, btype='low')
26
27     return b, a
28     #应用零相位滤波器（避免相位延迟），在通过低通滤波器筛去高频的噪声信号后消去相位延
    迟，与原始数据对齐
29 def apply_filter(price_series, b, a):
30     # 保留索引，避免错位
31     valid_idx = price_series.dropna().index
32     price_clean = price_series.loc[valid_idx].values#提取有效数据，纯数字的数组
33     filtered = filtfilt(b, a, price_clean)
34
35     # 返回带索引的序列
36     filtered_series = price_series.copy()#保留日期和结构
37     filtered_series.loc[valid_idx] = filtered#填回数据，保证对齐
38     return filtered_series
39     def apply_filter_to_stock(price_series, cutoff_period, order=4, filter_type='
        butterworth'):
40         """
41         股票专用：直接传入 周期参数，自动转频率 + 滤波
42         """
43         # 复制一份，避免修改原数据
44         price = price_series.copy()
45         # 股票核心：周期 → 频率 转换
46         if cutoff_period <= 1:
47             cutoff_period = 2 # 防止除0
48
49         cutoff_freq = 1.0 / cutoff_period # 周期转频率
50         fs = 1.0 # 日线：1天1个点
51
52         # 记录有效索引（非 NaN 位置）
53         valid_mask = price.notna()
54         price_valid = price[valid_mask] # 只拿干净数据滤波
55
56         if len(price_valid) == 0:
57             # 全是空值，返回全空序列
58             return pd.Series(index=price.index, dtype=price.dtype), None, None
59
60         # 设计滤波器
61         b, a = design_lowpass_filter(

```

```

62         cutoff_freq=cutoff_freq,
63         fs=fs,
64         order=order,
65         filter_type=filter_type
66     )
67
68     # 零相位滤波
69     filtered_clean = apply_filter(price_valid, b, a)
70
71     # 把滤波结果还原成带日期索引的Series (保持对齐)
72     filtered_series = pd.Series(index=price.index, dtype=price.dtype)
73     filtered_series.loc[valid_mask[valid_mask].index] = filtered_clean#填上滤波后的数据
74
75     return filtered_series, b, a
76
77     def evaluate_filter_performance(original, filtered, residual):#三个输入, 原始股价, 过滤后的股价和残差 (过滤后的噪声)
78         # 去无效值
79         mask = original.notna() & filtered.notna()#找到原始数据不为空且滤波数据也不为空的位置
80         orig = original[mask]
81         filt = filtered[mask]
82         res = residual[mask]
83
84         # 指标
85         #降噪率
86         #公式: 1 - 噪声 / 原始波动
87         noise_reduction = 1 - (np.var(res) / np.var(orig)) if np.var(orig) != 0 else 0
88         #平滑度比例
89         #滤波后波动 / 原始波动
90         smoothness_ratio = np.var(np.diff(filt)) / (np.var(np.diff(orig)) + 1e-8) if len(filt) > 1 else 0
91         #趋势保留度
92         #1 - 趋势相关系数
93         trend_correlation = np.corrcoef(orig, filt)[0, 1]
94         information_loss = 1 - trend_correlation#越接近1越好
95
96         #能量保留比例
97         energy_original = np.sum(orig ** 2) # 原始信号能量
98         energy_filtered = np.sum(filt ** 2) # 滤波后信号能量
99         energy_remaining = energy_filtered / (energy_original + 1e-8) # 能量保留比例
100
101     return {
102         'noise_reduction': round(noise_reduction, 4),
103         'smoothness_ratio': round(smoothness_ratio, 4),
104         'trend_correlation': round(trend_correlation, 4),

```

```

104         'information_loss': round(information_loss, 4),
105         'energy_remaining': round(energy_remaining, 4)
106     }
107     #绘制滤波器频率响应
108 def plot_filter_response(b, a, cutoff_freq, fs=1.0):
109     w, h = signal.freqz(b, a, worN=2000)#计算滤波器在所有频率上的响应,w: 角频率,h: 幅
        度响应 (强弱)
110     freq = w * fs / (2 * np.pi)#转换角频率
111
112     fig, ax = plt.subplots(figsize=(10, 4))
113     ax.plot(freq, 20 * np.log10(abs(h)), 'b-', linewidth=2)#画蓝色曲线 = 滤波器的幅度
        响应
114     ax.axvline(cutoff_freq, color='r', linestyle='--', label=f'截止频率: {cutoff_freq
        :.3f} Hz')
115     ax.set_xlabel('频率 (Hz)')
116     ax.set_ylabel('幅度 (dB)')
117     ax.set_title('滤波器频率响应')
118     ax.legend()
119     ax.grid(True, alpha=0.3)
120     ax.set_xlim(0, 0.5)
121
122     return fig
123     # 批量评估所有股票, 使用上面定义的函数
124 def batch_evaluate(stock_data_dict, filter_params_dict):
125     results = []
126     # 用于保存所有股票的滤波结果 (合并输出)
127     all_filtered_results = []
128     for code, data in stock_data_dict.items():
129         if code not in filter_params_dict:
130             continue
131
132         price = data.set_index('trade_date')['close']#把日期设为索引, 只拿 close 收盘
            价做滤波
133         params = filter_params_dict[code]#取出这只股票专属的滤波参数
134
135         # 应用滤波器
136         filtered, b, a = apply_filter_to_stock(
137             price,
138             cutoff_period=params['cutoff_period'],
139             order=params['filter_order'],
140             filter_type=params['filter_type']
141         )
142         #计算截止频率
143         cutoff_freq = 1.0 / params['cutoff_period']
144         # 绘制滤波器频率响应图
145         fig = plot_filter_response(b, a, cutoff_freq, fs=1.0)

```



```

146 fig.suptitle(f"股票 {code} | 分组 {params['group']} | 周期 {params['cutoff_period']}日", fontsize=12)
147 plt.tight_layout()
148 plt.savefig(f"../图表/滤波器响应图/滤波器响应图_{code}.png", dpi=150,
149           bbox_inches='tight')
150 plt.show() # 显示图片
151 plt.close(fig)
152
153 # 构建单只股票的完整数据表：日期 + 原始收盘价 + 滤波后收盘价 + 残差
154 stock_filter_df = pd.DataFrame({
155     'trade_date': price.index,
156     'code': code,
157     'close_price': price.values,
158     'filtered_price': filtered.values,
159     'residual': price.values - filtered.values # 残差 = 原始 - 滤波
160 })
161 # 把当前股票结果加入总表
162 all_filtered_results.append(stock_filter_df)
163
164
165 p = price.values
166 f = filtered.values
167
168 # 有效掩码（纯数组，无索引），解决price和filter索引对不齐的问题
169 valid_mask = ~(np.isnan(p) | np.isnan(f))
170
171 # 只取有效数据
172 p_valid = p[valid_mask]
173 f_valid = f[valid_mask]
174 residual_valid = p_valid - f_valid
175
176 # 传给评估函数（不需要索引也能计算）
177 metrics = evaluate_filter_performance(
178     pd.Series(p_valid),
179     pd.Series(f_valid),
180     pd.Series(residual_valid)
181 )
182
183 results.append({
184     'code': code,
185     'group': params['group'],
186     'cutoff_period': params['cutoff_period'],
187     'noise_reduction': metrics['noise_reduction'],
188     'smoothness_ratio': metrics['smoothness_ratio'],
189     'trend_correlation': metrics['trend_correlation'],

```

```

190         'information_loss': metrics['information_loss'],
191         'energy_remaining': metrics['energy_remaining']
192     })
193     if all_filtered_results:
194         all_filtered_df = pd.concat(all_filtered_results, ignore_index=True)
195         all_filtered_df.to_csv(r"../数据/全股票滤波结果总表.csv", index=False,
196                               encoding='utf-8-sig')
197         print("全股票滤波结果已保存到：全股票滤波结果总表.csv")
198
199     return pd.DataFrame(results)
200     filter_params_dict = filter_params_df.drop_duplicates('code').set_index('code').
201         to_dict(orient='index')
202     result_df = batch_evaluate(stock_data, filter_params_dict)
203     print("\n批量评估完成：")
204     print(result_df)
205     result_df.to_csv(r"../数据/评估效果.csv")

```

10.1.5 构建 DSP 因子

```

1     import numpy as np
2     import pandas as pd
3     import matplotlib.pyplot as plt
4     from pathlib import Path
5     # 设置中文字体
6     plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'WenQuanYi Micro Hei']
7     plt.rcParams['axes.unicode_minus'] = False
8     # filtered_series_dict: 包含每只股票的 original, filtered, group, params
9
10    def build_dsp_factor(original, filtered, lookback=60):
11        """
12        构建DSP增强因子
13
14        因子逻辑：
15        - 分子：滤波后价格的动量（趋势强度）
16        - 分母：噪声强度（残差的滚动波动率）
17        - 因子值高 = 趋势强 + 噪声低 = 高质量买入信号
18
19        参数：
20        - original: 原始价格序列
21        - filtered: 滤波后价格序列
22        - lookback: 动量计算周期（60天）
23
24        返回：
25        - dsp_factor: DSP增强因子
26        - filtered_momentum: 滤波后动量

```

```

27     - original_momentum: 原始动量 (对比用)
28     - noise_intensity: 噪声强度
29     """
30     # 对齐数据
31     common_idx = original.index.intersection(filtered.index)
32     orig_aligned = original.loc[common_idx]
33     filt_aligned = filtered.loc[common_idx]
34
35     # 1. 滤波后动量 (趋势强度)
36     filtered_momentum = filt_aligned.pct_change( periods=lookback )
37
38     # 2. 噪声强度 (残差的滚动波动率)
39     residual = orig_aligned.values - filt_aligned.values
40     residual_series = pd.Series( residual, index=common_idx )
41     noise_intensity = residual_series.rolling( window=lookback ).std()
42
43     # 3. DSP增强因子 = 动量 / 噪声强度
44     dsp_factor = filtered_momentum / ( noise_intensity + 1e-8 )
45
46     # 4. 原始动量 (作为对比基准)
47     original_momentum = orig_aligned.pct_change( periods=lookback )
48
49     return {
50         'dsp_factor': dsp_factor,
51         'filtered_momentum': filtered_momentum,
52         'original_momentum': original_momentum,
53         'noise_intensity': noise_intensity
54     }
55
56 # 1. 读取总表
57 df_all = pd.read_csv( r"..\数据\全股票滤波结果总表.csv" )
58
59 # 2. 把日期设为索引
60 df_all['trade_date'] = pd.to_datetime( df_all['trade_date'] )
61 df_all = df_all.set_index( 'trade_date' ).sort_index()
62
63 # 3. 按股票代码分组遍历
64 factors_dict = {}
65 factor_rows = [] # 用于保存总表
66 for code, df_stock in df_all.groupby( 'code' ):
67     # 取出当前股票: 原始收盘价 + 滤波后价格
68     original = df_stock['close_price'] # 正确字段名
69     filtered = df_stock['filtered_price'] # 正确字段名
70
71     # 构建DSP因子
72     factors = build_dsp_factor( original, filtered, lookback=60 )

```

```

73     factors_dict[code] = factors
74
75     # 拼合成一行行数据，方便存 CSV
76     temp = pd.DataFrame({
77         'code': code,
78         'close_price': original,
79         'filtered_price': filtered,
80         'dsp_factor': factors['dsp_factor'],
81         'filtered_momentum': factors['filtered_momentum'],
82         'original_momentum': factors['original_momentum'],
83         'noise_intensity': factors['noise_intensity']
84     })
85     factor_rows.append(temp)
86
87     # 输出统计信息
88     dsp_factor = factors['dsp_factor'].dropna()
89     if len(dsp_factor) > 0:
90         print(f"股票 {code} | DSP因子均值={dsp_factor.mean():8.4f} | 标准差={
          dsp_factor.std():8.4f}")
91
92 # 3. 保存完整因子总表
93 df_factor_all = pd.concat(factor_rows)
94 save_path = r"..\\数据\\DSP因子总表.csv"
95 df_factor_all.to_csv(save_path, encoding='utf-8-sig')
96 print("\n因子总表已保存至：")
97 print(save_path)
98
99 # 4. 批量画图：价格 + DSP因子 + 噪声强度
100 print("\n开始生成因子对比图...")
101 out_dir = Path(r"..\\图表\\因子图表")
102 out_dir.mkdir(exist_ok=True)
103
104 for code, df_stock in df_all.groupby('code'):
105     factors = factors_dict[code]
106
107     fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)
108     ax1, ax2, ax3 = axes
109
110     # 子图1：原始价 & 滤波价
111     ax1.plot(df_stock.index, df_stock['close_price'], label='原始收盘价', c='#1f77b4',
112             linewidth=1.2)
113     ax1.plot(df_stock.index, df_stock['filtered_price'], label='滤波趋势', c='#ff7f0e',
114             linewidth=2)
115     ax1.set_title(f'{code} 价格与滤波趋势', fontsize=14)
116     ax1.legend()
117     ax1.grid(alpha=0.3)

```

```

116
117 # 子图2: DSP增强因子
118 ax2.plot(factors['dsp_factor'], label='DSP因子', c='#2ca02c', linewidth=1.5)
119 ax2.axhline(0, c='red', linestyle='--', alpha=0.7)
120 ax2.set_title('DSP 增强因子 (趋势/噪声) ')
121 ax2.legend()
122 ax2.grid(alpha=0.3)
123
124 # 子图3: 噪声强度
125 ax3.plot(factors['noise_intensity'], label='噪声强度', c='#d62728', linewidth=1.2)
126 ax3.set_title('残差滚动波动率 (噪声强度) ')
127 ax3.legend()
128 ax3.grid(alpha=0.3)
129
130 plt.tight_layout()
131 fig.savefig(out_dir / f"因子图表_{code}.png", dpi=150, bbox_inches='tight')
132 plt.close(fig)
133
134 print("="*60)
135 print("全部完成!")
136 print(" - DSP因子总表已保存")
137 print(" - 每只股票三张对比图已生成")
138 print("="*60)

```

10.1.6 因子回测

```

1
2 # ----- 2. 计算未来收益 (用于因子评估) -----
3 forward_period = 5 # 未来5日收益
4 df['forward_return'] = df.groupby('code')['close_price'].apply(
5     lambda x: x.pct_change(forward_period).shift(-forward_period)
6 ).reset_index(level=0, drop=True)#计算未来五天的收益率波动
7
8 # ----- 3. 定义因子评估函数 -----
9 def evaluate_single_factor(df_stock, factor_col, forward_ret_col='forward_return',
10                             quantiles=5):
11     temp = df_stock[[factor_col, forward_ret_col]].copy()#保留因子值和未来收益两列
12     temp = temp.dropna()
13
14     if len(temp) < quantiles + 1:
15         return None#数据过少不评估
16
17     ic = temp[factor_col].corr(temp[forward_ret_col])#计算IC值
18     temp['group'] = pd.qcut(temp[factor_col].rank(pct=True), quantiles, labels=False)#
19     将因子值从低到高分成5组

```

```

18 group_ret = temp.groupby('group')[forward_ret_col].mean()#计算每组未来收益的平均值
19 top_ret = group_ret.iloc[-1]#因子值最大的一组
20 bottom_ret = group_ret.iloc[0]#最小的一组
21 long_short = top_ret - bottom_ret#做多最好组 + 做空最差组的收益
22 win_rate = (temp[temp['group'] == quantiles-1][forward_ret_col] > 0).mean()#买入
    Top 组后，赚钱的概率
23
24 # ===== 换手率计算 =====
25 top_group = temp[temp['group'] == quantiles - 1].copy()
26
27 # 个股自身换手率：连续两期是否在Top组
28 top_group['in_top'] = 1
29 top_group['prev_in_top'] = top_group['in_top'].shift(1)#上一期是否在top 里面
30 top_group['turnover'] = (top_group['prev_in_top'] != 1).astype(float)
31 #昨天不在top里，今天在，则为1
32 avg_turnover = top_group['turnover'].mean()
33
34 return {
35     'IC': ic,
36     'top_group_return': top_ret,
37     'long_short_return': long_short,
38     'win_rate': win_rate,
39     'turnover': avg_turnover
40 }
41
42 # ----- 4. 批量评估：DSP因子 vs 传统动量 -----
43 results = []
44 for code, group in df.groupby('code'):
45     res_dsp = evaluate_single_factor(group, 'dsp_factor')
46     res_trad = evaluate_single_factor(group, 'original_momentum')
47
48     if res_dsp is None or res_trad is None:
49         continue
50
51     results.append({
52         'code': code,
53         'IC_DSP': res_dsp['IC'],
54         'IC_Traditional': res_trad['IC'],
55         'long_short_DSP': res_dsp['long_short_return'],
56         'long_short_Traditional': res_trad['long_short_return'],
57         'win_rate_DSP': res_dsp['win_rate'],
58         'win_rate_Traditional': res_trad['win_rate'],
59         'turnover_DSP': res_dsp['turnover'],
60         'turnover_Traditional': res_trad['turnover'],
61     })
62

```

```
63 df_results = pd.DataFrame(results)
```

10.2 回测部分代码

10.2.1 行业市值中性化代码

```
1     import statsmodels.api as sm#线性回归工具
2
3 def neutralize_return_daily(df):
4     """
5     每天截面中性化：
6     对数收益率 log_return 剔除 行业 + 市值 的影响
7     输出：纯净收益率 log_return_neutral
8     """
9     def _neutralize(group):
10         # 1. 市值取 log
11         log_mv = np.log(group['market_cap'])
12
13         # 2. 行业哑变量，每一个行业赋值，便于识别
14         ind_dummies = pd.get_dummies(group['industry'], drop_first=True)
15
16         # 3. 构造回归 X
17         X = pd.concat([ind_dummies, log_mv.rename('log_mv')], axis=1)
18         X = sm.add_constant(X)
19
20         # 4. 回归原始收益率
21         y = group['log_return']
22
23         try:
24             # 残差 = 纯净收益率
25             res = sm.OLS(y, X).fit().resid
26         except:
27             res = y # 回归失败就用原值
28
29         group['log_return_neutral'] = res
30         return group
31
32     # 按【每天】做截面中性化（最标准）
33     return df.groupby('trade_date', group_keys=False).apply(_neutralize)
34
35 # 先计算原始 log_return
36 df_all['log_return'] = np.log(df_all['close'] / df_all['close'].shift(1))
37
38 # 执行中性化
39 df_all = neutralize_return_daily(df_all)
```

10.2.2 机器学习代码（全体 A 股）

```
1      # 1. 特征处理
2      df_noise['log_kurtosis'] = np.log1p(df_noise['峰度'])
3
4      # 用于机器学习的全部噪声特征
5      feature_cols = [
6          '收益率标准差（波动率）',
7          'log_kurtosis',
8          '一阶自相关系数',
9          '极端值比例',
10         '波动率聚集变异系数'
11     ]
12     df_noise = df_noise.dropna(subset=feature_cols).copy()
13     # 特征标准化
14     scaler = MinMaxScaler()
15     X = df_noise[feature_cols].copy()
16     X = scaler.fit_transform(X)
17
18     # -----
19     # 【AI 打分】随机森林自动学习噪声强度（不再人工定权重）
20     # -----
21     y = df_noise['收益率标准差（波动率）'] # 目标：预测噪声强度
22     rf = RandomForestRegressor(n_estimators=150, max_depth=6, random_state=42)
23     rf.fit(X, y)
24     df_noise['noise_score'] = rf.predict(X) # 模型自动算出的噪声得分
25
26     # -----
27     # 【AI 分组】KMeans 自动聚类分3组（不再人工三等分）
28     # -----
29     km = KMeans(n_clusters=3, random_state=42)
30     df_noise['cluster'] = km.fit_predict(X)
31
32     # 自动按得分高低映射：高噪声组 / 中噪声组 / 低噪声组
33     cluster_mean = df_noise.groupby('cluster')['noise_score'].mean().sort_values(ascending=False)
34     cluster_map = {
35         cluster_mean.index[0]: '高噪声组',
36         cluster_mean.index[1]: '中噪声组',
37         cluster_mean.index[2]: '低噪声组'
38     }
39     df_noise['group'] = df_noise['cluster'].map(cluster_map)
40
41
42     print("\n=== 【机器学习自动分组结果】 ===")
43     for g in ['高噪声组', '中噪声组', '低噪声组']:
```



```

44     sub = df_noise[df_noise['group'] == g]
45     print(f"\n{g} ({len(sub)}只) | 平均噪声得分: {sub['noise_score'].mean():.3f}")
46
47 print("\n=== 模型自动学习的特征重要性 ===")
48 for col, imp in zip(feature_cols, rf.feature_importances_):
49     print(f"{col}: {imp:.1%}")
50
51 # 保存 (格式不变)
52 df_noise[['code', 'group', 'noise_score', '收益率标准差 (波动率)',
53          '峰度', '一阶自相关系数']].to_csv(r'../数据/噪声分组.csv', index=False)
54     # 根据股票的噪声特征设计滤波器参数
55     """
56     返回:
57     - cutoff_freq: 截止频率 (归一化频率, 0-0.5)
58     - filter_order: 滤波器阶数
59     - filter_type: 滤波器类型
60     """
61     # 1. 训练特征
62     feature_cols = [
63         '收益率标准差 (波动率)',
64         'log_kurtosis',
65         '一阶自相关系数',
66         '极端值比例',
67         '波动率聚集变异系数']
68
69     X = df_noise[feature_cols].copy()
70     scaler = MinMaxScaler()
71     X_scaled = scaler.fit_transform(X)
72
73     # 【模型1】预测: 截止频率 cutoff_freq (回归任务)
74     # 目标变量: 用噪声得分自动生成最优目标
75     y_cutoff = 0.01 + 0.04 * (1 - df_noise['noise_score'] / df_noise['noise_score'].
76         max())
77
78     model_cutoff = RandomForestRegressor(n_estimators=100, random_state=42)
79     model_cutoff.fit(X_scaled, y_cutoff)
80     df_noise['cutoff_freq'] = model_cutoff.predict(X_scaled)
81     df_noise['cutoff_freq'] = df_noise['cutoff_freq'].clip(0.01, 0.06) # 安全范围
82
83     # -----
84     # 【模型2】预测: 滤波器阶数 filter_order (分类/整数预测)
85     # -----
86     y_order = np.clip(3 + (df_noise['峰度'] > 5).astype(int) + (df_noise['极端值比例'] >
87         0.01).astype(int), 3, 6)
88
89     model_order = RandomForestClassifier(n_estimators=100, random_state=42)

```

```

88 model_order.fit(X_scaled, y_order)
89 df_noise['filter_order'] = model_order.predict(X_scaled)
90
91 # -----
92 # 【模型3】预测：滤波器类型 filter_type (分类任务: butterworth / chebyshev)
93 # -----
94 y_type = np.where(df_noise['极端值比例'] > 0.015, 'butterworth', 'chebyshev')
95
96 model_type = RandomForestClassifier(n_estimators=100, random_state=42)
97 model_type.fit(X_scaled, y_type)
98 df_noise['filter_type'] = model_type.predict(X_scaled)
99
100 # -----
101 # 自动计算周期 (由截止频率导出)
102 # -----
103 df_noise['cutoff_period'] = (1 / df_noise['cutoff_freq']).round().astype(int)
104 df_noise['cutoff_period'] = df_noise['cutoff_period'].clip(15, 60)
105
106 # ===== 输出最终滤波器参数 =====
107 filter_df = df_noise[['code', 'group', 'cutoff_freq', 'cutoff_period',
108                      'filter_order', 'filter_type']].copy()
109
110 print("=== 机器学习全自动生成的滤波器参数 ===")
111 print(filter_df.head(10))
112
113 # 保存
114 filter_df.to_csv(r'../数据/滤波器参数设置.csv', index=False, encoding='utf-8-sig')

```